JPQL and Criteria API

Java Persistence Query Language | Criteria API | Spring Data

Chapter 04 - Section 04 | JPA / Hibernate

JPQL Basics	Entity/field names, not table/column names
Named vs Positional Params	:name vs ?1 parameter styles
JOIN FETCH	Eager-load associations in one query
@NamedQuery	Pre-compile JPQL on the entity class
Criteria API	Type-safe programmatic query building
Spring Data Query Methods	Derived names + @Query annotation
Native SQL	@Query(nativeQuery=true) for raw SQL

1 JPQL Basics

Entity and field names — not table/column names

JPQL (Java Persistence Query Language) looks like SQL but works on your Java objects instead of database tables. You write the name of the entity class and its fields, not the table name or column name. JPA translates the JPQL to SQL for you at runtime.

Java Example

```
// Entity: class Employee  ->  table: employees
// Field:  name            ->  column: emp_name

String jpql = "SELECT e FROM Employee e WHERE e.name = 'Alice'";
TypedQuery<Employee> query = em.createQuery(jpql, Employee.class);
List<Employee> result = query.getResultList();
```

Diagram: JPQL vs SQL Side-by-Side

Concept	SQL (database)	JPQL (Java objects)
FROM clause	FROM employees	FROM Employee e
Column filter	WHERE emp_name = 'Alice'	WHERE e.name = 'Alice'
Join	JOIN departments d ON ...	JOIN e.department d
Count	SELECT COUNT(*)	SELECT COUNT(e)
Update	UPDATE employees SET ...	UPDATE Employee e SET ...

JPQL always uses the Java class name and field name, never the SQL table/column name.

Key Rules

- Use the entity class name (Employee) not the table name (employees).
- Use field names (e.name) not column names (emp_name).
- JPQL is case-sensitive for class/field names but not for keywords.
- Always use TypedQuery to get compile-time type safety.
- em.createQuery() compiles JPQL every call; prefer @NamedQuery for repeated queries.

Q1. What does JPQL stand for?

Java Persistence Query Language. It is the object-oriented query language defined by JPA.

Q2. Why write Employee instead of employees in JPQL?

Because JPQL works on entity classes, not database tables. Employee is the class name.

Q3. How do you run a JPQL query?

Use EntityManager.createQuery(jpql, ResultType.class) and call getResultList() or getSingleResult().

Q4. Is JPQL portable across databases?

Yes. JPA translates JPQL to the correct SQL dialect for your database automatically.

Q5. What is TypedQuery and why use it?

TypedQuery is a generic query that guarantees the return type, preventing ClassCastException at runtime.

2 Named vs Positional Parameters

:name parameters vs ?1 positional parameters

JPQL supports two styles of bind parameters to protect against SQL injection and allow query reuse. Named parameters use a colon prefix (:paramName) and are bound by name. Positional parameters use a question mark and a number (?1) and are bound by position.

Java Example — Named Parameters (:name)

```
String jpql = "SELECT e FROM Employee e WHERE e.dept = :dept AND e.salary > :min";
TypedQuery<Employee> q = em.createQuery(jpql, Employee.class);
q.setParameter("dept", "Engineering");
q.setParameter("min", 50000.0);
List<Employee> list = q.getResultList();
```

Java Example — Positional Parameters (?1)

```
String jpql = "SELECT e FROM Employee e WHERE e.dept = ?1 AND e.salary > ?2";
TypedQuery<Employee> q = em.createQuery(jpql, Employee.class);
q.setParameter(1, "Engineering");
q.setParameter(2, 50000.0);
List<Employee> list = q.getResultList();
```

Key Rules

- Named parameters (:name) are preferred — more readable and order-independent.
- Positional parameters (?1) are numbered from 1, not 0.
- Never concatenate user input into a JPQL string; always use parameters.
- Spring Data @Query uses :name with @Param("name") or ?1 by argument position.
- Both styles prevent SQL injection because the value is never embedded in the query string.

Q1. What is the syntax for a named parameter in JPQL?

A colon followed by the parameter name, e.g., :salary or :departmentName.

Q2. What is the syntax for a positional parameter?

A question mark followed by a 1-based integer, e.g., ?1 for the first argument.

Q3. Which style is recommended and why?

Named parameters (:name) are recommended because they are self-documenting and order-independent.

Q4. How do you bind a named parameter programmatically?

Call query.setParameter("paramName", value) on the TypedQuery object.

Q5. Can you mix named and positional parameters in one query?

No. JPA requires you to use one style consistently within a single query.

3 JPQL JOIN FETCH

Eagerly load associations in a single query

By default JPA uses lazy loading for associations: related entities are loaded only when you access them, causing extra SQL queries (the N+1 problem). JOIN FETCH forces JPA to load the association in the same query, giving you all the data you need in one round trip.

Java Example

```
// Without JOIN FETCH: loads Department lazily (extra query per employee)
String jpql1 = "SELECT e FROM Employee e";

// With JOIN FETCH: loads Department eagerly in the same SQL JOIN
String jpql2 = "SELECT e FROM Employee e JOIN FETCH e.department d";

TypedQuery<Employee> q = em.createQuery(jpql2, Employee.class);
List<Employee> employees = q.getResultList();
// e.getDepartment() is already loaded — no extra SQL fired
```

Key Rules

- JOIN FETCH eliminates the N+1 query problem for associations.
- Use JOIN FETCH only when you actually need the related data.
- You can chain: JOIN FETCH e.department d JOIN FETCH d.manager.
- JOIN FETCH cannot be combined with setMaxResults easily; use a subquery instead.
- LEFT JOIN FETCH loads the parent even when the association is null/empty.

Q1. What problem does JOIN FETCH solve?

The N+1 query problem, where JPA fires one extra SQL query per row to load a lazy association.

Q2. What is the difference between JOIN and JOIN FETCH?

JOIN filters rows but does not initialise the association. JOIN FETCH loads the association eagerly.

Q3. When should you avoid JOIN FETCH?

When you do not need the associated data in that request, to avoid loading unnecessary objects.

Q4. What does LEFT JOIN FETCH do differently?

It includes the owning entity even when the association collection is empty or null.

Q5. How does JOIN FETCH affect the SQL generated?

JPA emits a SQL JOIN that selects columns from both tables in a single database round trip.

4 @NamedQuery

Pre-compiled, reusable JPQL declared on the entity

@NamedQuery lets you declare a JPQL query as a constant on the entity class. The query is validated and compiled once at application startup, catching syntax errors early. Named queries are referenced by their logical name at runtime.

Java Example

```
@Entity
@NamedQuery(
    name  = "Employee.findByDept",
    query = "SELECT e FROM Employee e WHERE e.dept = :dept"
)
public class Employee {
    @Id Long id;
    String name;
    String dept;
    double salary;
}

// Usage:
List<Employee> list = em
    .createNamedQuery("Employee.findByDept", Employee.class)
    .setParameter("dept", "HR")
    .getResultList();
```

Key Rules

- Name queries as EntityClass.queryPurpose to avoid collisions.
- JPA validates @NamedQuery at startup — errors surface immediately.
- Use @NamedQueries({...}) to declare multiple queries on one entity.
- Named queries are globally unique within the persistence unit.
- Spring Data @Repository methods can delegate to @NamedQuery automatically.

Q1. When is a @NamedQuery compiled?

At application startup, during EntityManagerFactory initialisation, not at query execution time.

Q2. What is the benefit of early compilation?

JPQL syntax errors are caught at startup rather than at runtime during a user request.

Q3. How do you declare multiple named queries on one entity?

Wrap them in @NamedQueries({@NamedQuery(...), @NamedQuery(...)}) on the class.

Q4. What naming convention is recommended?

EntityClassName.queryDescription, for example Employee.findByDepartment.

Q5. How do you execute a named query?

Call em.createNamedQuery("QueryName", ReturnType.class) on the EntityManager.

5 Criteria API

Type-safe programmatic query construction

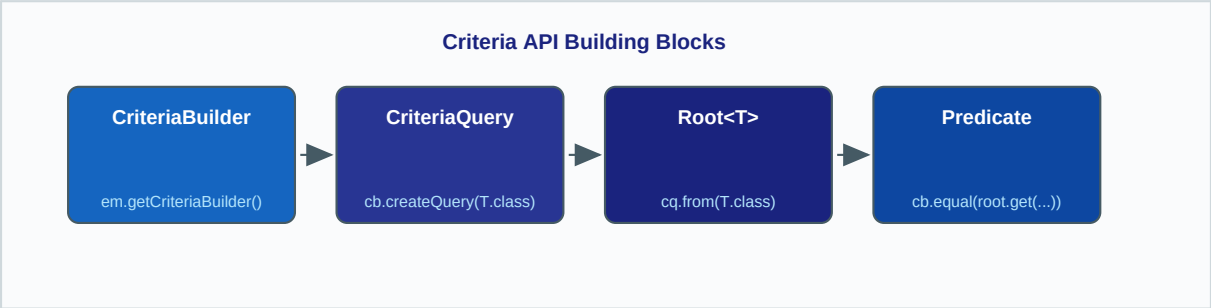
The Criteria API lets you build queries using Java method calls instead of strings. This gives you compile-time type checking — a typo in a field name is a compile error, not a runtime exception. It is especially useful when query conditions are dynamic.

Java Example

```
CriteriaBuilder   cb  = em.getCriteriaBuilder();
CriteriaQuery<Employee> cq  = cb.createQuery(Employee.class);
Root<Employee>    root = cq.from(Employee.class);

Predicate dept = cb.equal(root.get("dept"), "Engineering");
Predicate sal  = cb.greaterThan(root.get("salary"), 50000.0);

cq.select(root).where(cb.and(dept, sal));
List<Employee> result = em.createQuery(cq).getResultList();
```



CriteriaBuilder creates CriteriaQuery, then Root defines the entity, then Predicates filter results.

Key Rules

- CriteriaBuilder is the factory — get it from EntityManager.getCriteriaBuilder().
- Root represents the entity you are querying (like the FROM clause).
- Predicates are conditions; combine them with cb.and() and cb.or().
- Use Metamodel (Employee_.name) for fully type-safe field references.
- Criteria API is verbose but ideal for dynamic queries with optional filters.

Q1. What is CriteriaBuilder used for?

It is a factory object that creates CriteriaQuery, Root, and Predicate objects.

Q2. What does Root represent in a Criteria query?

The entity being queried, equivalent to the FROM clause in JPQL.

Q3. What is a Predicate in the Criteria API?

A condition (like a WHERE clause expression) created via methods such as cb.equal() or cb.greaterThan().

Q4. When is the Criteria API better than JPQL?

When query conditions are dynamic and known only at runtime, since you build the query with if-statements.

Q5. What is the Metamodel and why use it?

Generated classes (Employee_) that represent entity fields as type-safe attributes, eliminating string-based field references.

6 Spring Data Query Methods vs @Query

Derived method names and explicit JPQL annotation

Spring Data JPA can generate queries automatically from repository method names. If the generated query is not powerful enough, use the @Query annotation to write JPQL (or native SQL) directly on the method.

Java Example — Derived Method Names

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Spring Data generates: SELECT e FROM Employee e WHERE e.dept = ?1
    List<Employee> findByDept(String dept);

    // AND + ORDER BY
    List<Employee> findByDeptAndSalaryGreaterThanOrderBySalaryDesc(
        String dept, double minSalary);
}
```

Java Example — @Query Annotation

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query("SELECT e FROM Employee e WHERE e.salary > :min ORDER BY e.salary DESC")
    List<Employee> findHighEarners(@Param("min") double minSalary);

    @Query("SELECT AVG(e.salary) FROM Employee e WHERE e.dept = :dept")
    Double avgSalaryForDept(@Param("dept") String dept);
}
```

Key Rules

- Derived method names work well for simple 1-2 condition queries.
- Very long derived names become hard to read — prefer @Query for complex queries.
- @Query JPQL binds :param via @Param("param") or ?1 by argument position.
- @Query methods support projections: return interfaces with getX() methods.
- Add @Modifying + @Transactional to @Query methods that UPDATE or DELETE.

Q1. How does Spring Data generate a query from a method name?

It parses the method name using keywords like findBy, And, Or, GreaterThan, OrderBy, etc.

Q2. When should you use @Query instead of a derived method name?

When the query is complex, involves joins or aggregates, or the method name would be too long.

Q3. How do you pass parameters in a @Query JPQL method?

Use @Param("name") on the method argument and reference it as :name in the query string.

Q4. What extra annotations are needed for a @Query that modifies data?

@Modifying and @Transactional must be added so Spring Data knows it is a write operation.

Q5. Can @Query return a single value like a count or average?

Yes. Declare the return type as Long, Double, Integer, etc., and Spring Data maps the scalar result.

7 Native SQL with @Query(nativeQuery=true)

Raw SQL queries when JPQL is not enough

Sometimes you need database-specific SQL features that JPQL does not support — window functions, CTEs, or vendor-specific functions. Adding nativeQuery=true to @Query tells Spring Data to send the string as raw SQL directly to the database.

Java Example

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Raw SQL — uses table name 'employees', not entity name 'Employee'
    @Query(value = "SELECT * FROM employees WHERE dept = :dept ORDER BY salary DESC",
           nativeQuery = true)
    List<Employee> findByDeptNative(@Param("dept") String dept);

    // Vendor-specific: use LIMIT (MySQL/PostgreSQL)
    @Query(value = "SELECT * FROM employees ORDER BY salary DESC LIMIT 5",
           nativeQuery = true)
    List<Employee> top5Earners();
}
```

Key Rules

- Native queries use SQL table/column names, not entity/field names.
- nativeQuery=true bypasses JPA translation — you own the SQL.
- Native queries are database-specific and reduce portability.
- For pagination with native queries, also set countQuery in @Query.
- Prefer JPQL or Criteria API; use native SQL only when JPQL cannot express the query.

Q1. What does nativeQuery=true tell Spring Data?

It tells Spring Data to send the query string as raw SQL to the database without JPQL parsing.

Q2. Do native queries use entity names or table names?

Table names (and column names) from the actual database schema, not the Java entity names.

Q3. What is the main drawback of native SQL queries?

They are tied to a specific database dialect, reducing portability if you switch databases.

Q4. How do you add pagination to a native @Query?

Add a countQuery attribute to @Query with the SQL to count total rows, then accept a Pageable argument.

Q5. When is nativeQuery=true the right choice?

When the query uses database-specific features like CTEs, window functions, or vendor-specific functions.